

Tutorato 1: Teoria e Basi dell'Assembly RISC-V

1 Domande a risposta multipla (Teoria)

Esercizio 1

Quanti registri generali possiede l'architettura base del RISC-V a 64 bit e qual è la loro dimensione?

- (A) 16 registri da 32 bit
- (B) 32 registri da 64 bit
- (C) 64 registri da 32 bit
- (D) 32 registri da 32 bit

Soluzione: (B) 32 registri da 64 bit (i registri `x0-x31` contengono *double word*).

Esercizio 2

Qual è lo scopo principale del registro `x0` nell'architettura RISC-V?

- (A) Viene utilizzato come Program Counter (PC).
- (B) Punta alla cima dello stack (Stack Pointer).
- (C) Contiene sempre il valore costante 0 e non può essere modificato.
- (D) Contiene l'indirizzo di ritorno per le chiamate a procedura.

Soluzione: (C) Contiene sempre la costante 0. Il RISC-V dedica un registro *ad hoc* allo 0 perché è di grande utilità per semplificare diverse operazioni.

Esercizio 3

Quale formato di istruzione viene utilizzato in RISC-V per eseguire operazioni che richiedono un operando costante (ad esempio l'istruzione `addi`)?

- (A) Formato R (Registro)
- (B) Formato I (Immediato)
- (C) Formato S (Store)
- (D) Formato J (Jump)

Soluzione: (B) Formato I. Viene usato per le istruzioni con immediati e di caricamento in memoria, permettendo di codificare una costante all'interno della singola word a 32 bit dell'istruzione.

Esercizio 4

Considerando l'ordine di memorizzazione dei byte in memoria (endianness), quale convenzione utilizza l'architettura RISC-V vista nel corso?

- (A) Big Endian (il byte più significativo all'indirizzo più basso)
- (B) Little Endian (il byte meno significativo all'indirizzo più basso)
- (C) Middle Endian
- (D) L'ordine dei byte è specificato dal programmatore tramite uno pseudo-codice.

Soluzione: (B) Little Endian.

2 Traduzione in Assembly RISC-V

Esercizio 5

Tradurre la seguente espressione C in Assembly RISC-V. Si assuma che le variabili **a**, **b**, **c**, **d** siano assegnate rispettivamente ai registri **x10**, **x11**, **x12**, **x13**. Utilizzare un registro temporaneo a scelta per i passaggi intermedi.

$a = (b + c) - d;$

```
add x5, x11, x12    % Uso il registro temporaneo x5 per calcolare (b + c)
sub x10, x5, x13     % Sottraggo d (x13) dalla somma e salvo il risultato in a (x10)
```

Esercizio 6

Tradurre la seguente istruzione di accesso in memoria ad un array in Assembly RISC-V. Si assuma che l'indirizzo base dell'array di *double word* **A** sia contenuto in **x22** e che la variabile **h** sia in **x21**.

$A[4] = h + A[2];$

Nel RISC-V l'indirizzamento è al byte. Poiché un elemento dell'array è una double word (8 byte), l'indice 2 corrisponde all'offset $2 \times 8 = 16$ e l'indice 4 corrisponde all'offset $4 \times 8 = 32$.

```
ld x9, 16(x22)      % Carica A[2] nel registro temporaneo x9
add x9, x21, x9       % Somma h (x21) con A[2] (x9)
sd x9, 32(x22)       % Salva il risultato all'indirizzo di A[4]
```

Esercizio 7

Tradurre la seguente operazione C sfruttando in modo efficiente le operazioni logiche del processore RISC-V (senza usare l'istruzione `mul`). Si assuma che **a** sia in **x10** e **b** sia in **x11**.

$a = b * 8;$

Moltiplicare per 8 equivale a moltiplicare per 2^3 . Si può quindi utilizzare uno shift logico a sinistra di 3 posizioni.

```
slli x10, x11, 3     % Esegue a = b « 3, che equivale ad a = b * 8
```

Esercizio 8

Tradurre il seguente blocco condizionale **if-else** in assembly RISC-V. Si assumo che le variabili *i*, *j*, *f*, *g*, *h* siano assegnate rispettivamente ai registri *x22*, *x23*, *x19*, *x20*, *x21*.

```
if (i < j)
    f = g + h;
else
    f = g - h;
```

Per tradurre il costrutto, si valuta la condizione opposta rispetto a quella dell'**if** per saltare al ramo **else**.

```
bge x22, x23, ELSE    % Salta a ELSE se i >= j (condizione opposta a i < j)
add x19, x20, x21     % f = g + h
beq x0, x0, ESCI      % Salto incondizionato alla fine del costrutto (ESCI)
ELSE:
sub x19, x20, x21     % f = g - h
ESCI:                 % Etichetta di fine blocco
```

Esercizio 9

Tradurre il seguente semplice costrutto **if** in assembly RISC-V. Si assumo che le variabili *i* e *j* siano assegnate ai registri *x22* e *x23*, e la variabile *f* sia in *x19*.

```
if (i == j) {
    f = f + 1;
}
```

Per tradurre un **if**, si valuta solitamente la condizione opposta per saltare oltre il blocco di codice.

```
bne x22, x23, FINE    % Se i != j, salta all'etichetta FINE
addi x19, x19, 1      % Esegue f = f + 1 usando un valore immediato
FINE:                 % Etichetta di fine blocco
```

Esercizio 10

Tradurre il seguente blocco `if-else` con un controllo di minoranza. Si assuma che `i` sia in `x22`, `j` in `x23`, `f` in `x19`, `g` in `x20` e `h` in `x21`.

```
if (i < j)
    f = g;
else
    f = h;
```

Anche qui usiamo la condizione opposta (maggiore o uguale) per saltare al ramo `else`. Per copiare un registro in un altro, possiamo sommarlo al registro `x0` che vale sempre zero.

```
bge x22, x23, ELSE    % Se i >= j salta a ELSE
add x19, x20, x0      % f = g (sommando g a 0)
beq x0, x0, FINE      % Salto incondizionato alla fine
ELSE:
add x19, x21, x0      % f = h (sommando h a 0)
FINE:
```

Esercizio 11

Tradurre la seguente operazione condizionale che usa una disuguaglianza. Si assuma che le variabili `i` e `j` siano assegnate ai registri `x22` e `x23`.

```
if (i != j) {
    i = i + 5;
}
```

L'opposto di diverso (`!=`) è uguale (`==`). Usiamo l'istruzione `beq` per saltare il blocco se la condizione è falsa.

```
beq x22, x23, FINE    % Se i == j salta all'etichetta FINE
addi x22, x22, 5      % i = i + 5
FINE:
```

Esercizio 12

Tradurre il seguente codice che esegue un calcolo e poi un semplice controllo sul risultato. Assumere che `a` sia in `x10`, `b` in `x11` e `c` in `x12`.

```
a = b + c;
if (a < 0) {
    a = 0;
}
```

Per verificare se un numero è minore di 0, lo si confronta direttamente con il registro `x0`. L'opposto di minore è maggiore o uguale (`bge`).

```
add x10, x11, x12    % a = b + c
bge x10, x0, FINE     % Se a >= 0 salta a FINE
add x10, x0, x0       % a = 0 (sommando 0 a 0)
FINE:
```

Esercizio 13

Tradurre il seguente ciclo `while` in assembly RISC-V. Si assuma che l'indice `i` sia nel registro `x22`, la variabile `k` in `x24` e l'indirizzo base dell'array `salva` sia in `x25`. L'array è composto da *double word*.

```
while (salva[i] == k)
    i += 1;
```

CICLO:

```
slli x10, x22, 3      % Moltiplica l'indice i per 8 (23) per ottenere lo spiazzamento in byte
add x10, x10, x25      % Aggiunge lo spiazzamento all'indirizzo base in x25
ld x9, 0(x10)         % Carica il valore di salva[i] nel registro temporaneo x9
bne x9, x24, ESCI     % Se salva[i] != k, esci dal ciclo
addi x22, x22, 1       % Incrementa l'indice i di 1
beq x0, x0, CICLO     % Salto incondizionato per ripetere il ciclo
ESCI:                 % Uscita dal ciclo
```